TestGen: Specification-Based Mutation Testing
Across Three Web Application Backends

Restaurant · E-Commerce · Library Web Applications

Backends Tested	3 (Node.js/Express + Java Spring Boot)
Total Bugs Injected	24 (8 per backend)
Total Detected	21 / 24 (87.5%)
Date	April 2026

Table of Contents

1. Executive Summary

This report documents mutation testing across three web application backends using **TestGen** — a C++17 framework that generates concrete API tests from formal pre/postcondition specifications. 24 mutation bugs were injected across three backends; TestGen detected 21 of them (87.5%).

Backend	Technology	Bugs	Detected	Score
Restaurant	Node.js / Express + MongoDB	8	5	62.5%
E-Commerce	Node.js / Express + MongoDB	8	8	100%
Library	Java Spring Boot + MySQL	8	8	100%
TOTAL	—	24	21	87.5%

2. TestGen Core Components

TestGen ATC (Abstract Test Case)

A program generated from the formal spec. It contains **assume()** statements (preconditions — what must be true before the API call) and **assert()** statements (postconditions — what must be true after the API call). The ATC is the bridge between the human-written spec and executable tests.

genATC

The module that compiles spec blocks into ATCs. It reads the spec file, extracts each operation's pre/postconditions, and produces the ATC program to be executed by the SEE.

SEE — Symbolic Execution Engine

Runs the ATC program twice. First iteration: all inputs are **symbolic (unknown)** variables. The SEE accumulates path constraints from all assume() statements it encounters. Second iteration: runs again with Z3-assigned concrete values, making real HTTP calls.

Z3 SMT Solver

Takes all path constraints collected by the SEE and finds a concrete assignment satisfying them all. If **UNSAT** (no assignment possible) → this test path is infeasible, skip it. If **SAT** → concrete test inputs found, proceed to backend execution.

Rewrite Globals

Transforms global map operations in the ATC (e.g., B[k]=v, in(x, dom(B))) into live test-API calls: **get_B(), set_B()**. This links the spec's abstract state assertions to the real live database state at runtime.

CTC — Concrete Test Case

The final executable test produced after Z3 solving. Contains: Z3-assigned input values, real HTTP calls to the running backend, and assert() evaluated against actual HTTP responses and DB state. **ASSERT FAILED = bug detected.**

3. ATC → CTC Pipeline

Stage 1 — SPEC → ATC

Formal spec with assume()/assert() statements compiled into Abstract Test Case program by genATC.

Stage 2 — Rewrite Globals

Global map ops (B[k]=v, in(x,dom(B))) rewritten to live get_B()/set_B() test-API calls that fetch real DB state.

Stage 3 — SEE Iteration 1 (Symbolic)

SEE executes ATC with symbolic variables. Path constraints from all assume() statements accumulated.

Stage 4 — Z3 Constraint Solving

Z3 finds concrete assignments satisfying all constraints. UNSAT = infeasible. SAT = concrete inputs ready.

Stage 5 — Backend Execution (Concrete)

SEE runs again with Z3 values. Real HTTP calls made to live backend. assert() evaluated against responses.

Stage 6 — CTC Output

Concrete Test Case recorded: inputs, HTTP calls, DB state, assertion outcome. FAILED = bug detected.

Bug detection example: deleteBook spec asserts not_in(bookCode, dom(B')). get_B() fetches live DB. If the book still exists after the delete call → not_in() = false → ASSERT FAILED → bug detected.

4. Restaurant Backend — Results & Bug Descriptions

Node.js/Express application on port 5002 (MongoDB). First backend tested with TestGen.

Bug	Location	Description	Bug Type	Det.	Depth
RB1	routes/auth.js	Registration returns HTTP 200 instead of 201	Wrong status code	NO ✗	—
RB2	routes/restaurants.js	Restaurant record not removed after deletion	Missing side-effect	YES ✓	2
RB3	routes/menu.js	Menu item creation returns HTTP 200 instead of 201	Wrong status code	NO ✗	—
RB4	routes/orders.js	Order status not updated after completion	Missing side-effect	YES ✓	3
RB5	routes/cart.js	Cart item not cleared after checkout	Missing side-effect	YES ✓	2
RB6	routes/reviews.js	Review creation returns HTTP 200 instead of 201	Wrong status code	NO ✗	—
RB7	models/Cart.js	Cart stores wrong quantity value (wrong field mapping)	Wrong field value	YES ✓	2
RB8	routes/restaurants.js	Restaurant record not correctly updated on PUT	Missing side-effect	YES ✓	2

Mutation Score: 5 / 8 (62.5%) — RB1, RB3, RB6 all changed HTTP 201 → 200 on creation responses. Undetectable because the factory accepts both 200 and 201 as success. This finding informed the design of all subsequent bug injections.

5. E-Commerce Backend — Results & Bug Descriptions

Node.js/Express application on port 3000 (MongoDB). All 8 bugs designed as state-change mutations — detectable via postcondition assertions regardless of HTTP status codes.

Bug	Location	Description	Bug Type	Det.	Depth
EB1	authController.js	User registered but not saved to DB — id=null returned	Save without persist	YES ✓	1
EB2	productController.js	Product created but not saved to DB — id=null returned	Save without persist	YES ✓	1
EB3	orderController.js	Order status field set to wrong value after creation	Wrong field value	YES ✓	2
EB4	cartController.js	Cart quantity doubled — item added with qty×2 instead of qty	Wrong field value	YES ✓	2
EB5	reviewController.js	Review submitted but not saved to DB	Missing side-effect	YES ✓	2
EB6	orderController.js	Total = sum of prices only, ignores quantity multiplier	Wrong calculation	YES ✓	2
EB7	productController.js	Buyer role can delete products (seller-only check bypassed)	Auth bypass	YES ✓	2
EB8	cartController.js	Add to cart succeeds even when quantity exceeds stock	Wrong guard	YES ✓	2

Mutation Score: 8 / 8 (100%) — EB2 cascade: product created with id=null cascades into all downstream tests (orders, cart, reviews) that require a valid product.

6. Library Backend — Results & Bug Descriptions

Java Spring Boot application on port 8080 (MySQL). Manages books, students, loan requests, and loan accept/return workflows. Most complex backend — different tech stack from the others.

Bug	Location	Description	Bug Type	Det.	Depth
LB1	RequestController.java:167	After loan accepted, original request NOT updated to ACCEPTED	Missing side-effect	YES ✓	4
LB2	BookService.java:43	deleteBook returns book without calling deleteById() — book stays in DB	Missing side-effect	YES ✓	2
LB3	RequestController.java:147	Overlap check inverted: throws error for valid non-overlapping requests	Wrong condition	YES ✓	4
LB4	BookStudentService.java:70	returnBook calls findById instead of deleteById — loan stays in DB	Missing side-effect	YES ✓	5
LB5	StudentService.java:50	deleteStudent calls getStudentById instead of deleteStudentById — student stays in DB	Missing side-effect	YES ✓	2
LB6	BookService.java:57-60	updateBook calls deleteById instead of updating — book deleted on 'update'	Wrong operation	YES ✓	2
LB7	StudentService.java:40	saveStudent returns input student without calling studentDAO.save() — id=0, not in DB	Missing side-effect	YES ✓	1
LB8	BookService.java:33	addBook returns new Book() without calling bookDAO.save() — id=0, not in DB	Missing side-effect	YES ✓	1

Mutation Score: 8 / 8 (100%) — No new spec blocks needed; existing LibrarySpec.cpp postconditions were sufficient to catch all 8 bugs. Deepest test chain: LB4 at depth=5 (saveBook → saveStudent → saveRequest → acceptRequest → returnBook).

7. Bug Test File Walkthrough

Two representative bugs are walked through in full detail to illustrate the complete TestGen pipeline: a simple depth=1 case (LB8) and a complex depth=4 chained case (LB3).

7.1 LB8 — Book Not Saved (depth=1, simplest case)

What was tested

Bug injected in BookService.java line 33: **return this.bookDAO.saveBook(book)** was replaced with **return new Book();** — the book is never persisted to the MySQL database.

Spec postcondition

```
saveBookOk: in(_result, dom(B')) // result ID must exist in live Books table
```

Pipeline of execution

Step	Action	Output
SEE iter 1	Symbolic var for bookTitle	Constraint accumulated
Z3	Find concrete bookTitle value	bookTitle = 'Test Book 6'
HTTP	POST /books/save with concrete title	Response: HTTP 200, bookCode=0
SaveBookFunc	Parse response bookCode	Returns String('0') — new Book() has code=0
get_B()	Fetch Books table from live DB	(empty) — no book with code=0 in DB
assert()	in('0', dom(B'))	false → ASSERT FAILED → BUG DETECTED

Why it works: Spring Boot's default int field bookCode is 0 when uninitialized. new Book() returns an object with bookCode=0. get_B() confirms no book with code=0 exists in the DB. The in-domain assertion catches it instantly at depth=1.

7.2 LB3 — Overlap Check Inverted (depth=4, complex chain)

What was tested

Bug injected in RequestController.java line 147: **if (doesRequestOverlap(request)) { throw }** was replaced with **if (!doesRequestOverlap(request)) { throw }** — the guard is inverted.

Spec postcondition

```
acceptRequestOk: AND(in(_result3, dom(Loans')), not_in(requestId, dom(Req'))) // accepted
request must become a loan AND be removed from requests
```

Pipeline of execution

Step	API Call	Result
1 — saveBookOk	POST /books/save	HTTP 201, bookCode = 42
2 — saveStudentOk	POST /student/save	HTTP 200, studentId = 7
3 — saveRequestOk	POST /request/save	HTTP 200, requestId = 15
4 — acceptRequestOk	POST /bookStudent/accept	HTTP 500 — exception thrown!

get_Loans()	Fetch live DB	loanId absent (accept failed)
assert()	in(500, dom(Loans'))	false → ASSERT FAILED → BUG DETECTED

Why it works: A fresh request has no competing loans, so doesRequestOverlap() = false. The inverted guard (!false = true) throws UnavailableForGivenDatesException → HTTP 500. AcceptRequestFunc receives 500 and returns Num(500) as the result. in(500, dom(Loans')) fails because 500 is not a loan ID.

8. Comparative Analysis

8.1 Detection by Bug Type

Bug Type	Total	Detected	Rate	Notes
Missing side-effect	11	11	100%	Core TestGen strength
Save without persist	3	3	100%	Caught at depth=1
Wrong operation	2	2	100%	State change catches it
Wrong conditional	2	2	100%	Exception → wrong HTTP → fails assert
Wrong field/calculation	3	3	100%	Spec value assertion
Auth bypass	1	1	100%	Role check postcondition
Wrong status code	3	0	0%	Factory accepts 200 & 201 both

8.2 Detection Depth Distribution

Depth	Count	Bugs
1	4	EB1, EB2, LB7, LB8 — save-without-persist, caught immediately
2	12	Most restaurant, ecommerce, and library delete/update bugs
3	1	RB4 — order status update
4	2	LB1, LB3 — require full book+student+request+accept chain
5	1	LB4 — deepest: book+student+request+accept+return

9. Key Takeaways

Postcondition state assertions are the primary detection mechanism

TestGen checks live DB state after each API call. If the expected state change didn't happen (record not deleted, record not saved, wrong record modified) → postcondition fails → bug detected. Works across Node.js+MongoDB and Java+MySQL without changing the assertion logic.

Status code mutations escape detection when factories are flexible

Factories designed to accept both 200 and 201 cannot detect status code bugs. All 3 missed restaurant bugs (RB1, RB3, RB6) were 201→200 mutations. Lesson: design bugs as state-change mutations for reliable detection.

Save-without-persist bugs are caught immediately at depth=1

return new Book() gives bookCode=0; return student gives id=0. Neither ID exists in DB. The in-domain assertion in(_result, dom(B')) fails instantly. These are the easiest bugs to detect — minimal test sequence needed.

Cascade detection amplifies bug signal

A single save-without-persist bug cascades into all downstream tests requiring that entity. LB8 (book not saved) → cascades into LB1, LB3, LB4, LB6 tests. LB7 (student not saved) → cascades into LB1, LB3, LB4 tests. Same pattern as EB2 in ecommerce.

Existing specs are often sufficient — no new blocks needed

Library backend: all 8 bugs detected with 0 new spec blocks added. Well-written postconditions covering save/delete/update/accept/return operations already express the right state invariants to catch new bugs.

TestGen scales across technology stacks

Successfully adapted from Node.js+MongoDB to Java Spring Boot+MySQL. Only the FunctionFactory and HTTP interaction layer needed adjustment. SEE, Z3, assertion evaluation, and the ATC→CTC pipeline are fully backend-agnostic.

10. Appendix — Output Files

All files in: /Users/nakulpanwar/Desktop/TestgenTool/all_test_files/

Restaurant Backend

- bugtest_B1-B8_*.txt
- bugtest_SUMMARY_ALL8_BUGS.txt

E-Commerce Backend

- bugtest_ECOM_EB1_user_not_saved.txt
- bugtest_ECOM_EB2_product_not_saved.txt
- bugtest_ECOM_EB3_order_wrong_status.txt
- bugtest_ECOM_EB4_cart_quantity_doubled.txt
- bugtest_ECOM_EB5_review_not_saved.txt
- bugtest_ECOM_EB6_wrong_order_total.txt
- bugtest_ECOM_EB7_buyer_deletes_product.txt
- bugtest_ECOM_EB8_cart_exceeds_stock.txt
- bugtest_ECOM_SUMMARY_ALL8_BUGS.txt

Library Backend

- bugtest_LIB_LB1_request_not_deleted_after_accept.txt
- bugtest_LIB_LB2_book_not_deleted.txt
- bugtest_LIB_LB3_overlap_check_inverted.txt
- bugtest_LIB_LB4_loan_not_deleted_on_return.txt
- bugtest_LIB_LB5_student_not_deleted.txt
- bugtest_LIB_LB6_update_deletes_book.txt
- bugtest_LIB_LB7_student_not_saved.txt
- bugtest_LIB_LB8_book_not_saved.txt
- bugtest_LIB_SUMMARY_ALL8_BUGS.txt